



INSTITUTO POLITÉCNICO NACIONAL

Unidad Profesional Interdisciplinaria de Ingeniería
y Ciencias Sociales y Administrativas



Reportes Tercer Departamental

Equipo: 3

Materia: Abstracción y uso de datos

Secuencia: 3CM30

Maestro: Yventz Entzaña Garduño

Semestre: Tercero

Integrantes:	Núm.lista.
• Enríquez Sánchez Ximena Noemí.....	6
• Guerrero Pérez Juan Marcos.....	15
• Reyes Reyes Daniel.....	34
• Tinoco Arriaga Mildred Elizabeth.....	39

Link de GitHub: <https://github.com/Mily300/Tercer-Parcial.git>

ÍNDICE

INTRODUCCIÓN.....	4
PRIMER PARCIAL.....	6
Práctica 1: Hola Mundo.....	6
Práctica 2: Suma de Dos Números como Parámetros.....	6
Práctica 3: Calculadora Básica.....	7
Práctica 4: Calculadora con Sobrecarga de Funciones.....	7
Práctica 5: Herencia.....	8
Práctica 6: Herencia y Sobreescritura.....	8
Práctica 7: Creación de Tipo de Dato (Auto-Persona).....	9
Práctica 7 Bis: Identificar Tamaños.....	10
Práctica 8: Promedio.....	10
Práctica 9: Promedio con arreglo.....	10
Práctica 10: Promedio con puntero al arreglo.....	11
Práctica 11: Matrices.....	12
Práctica 12: Nuevo tipo de dato como arreglo.....	12
Práctica 13: Nuevo tipo de dato con puntero al arreglo.....	13
Práctica 14: Recursividad.....	14
Practica 15: Fractales Recursivos (Polvo de Cantor y Triángulo de Sierpinski).....	14
SEGUNDO PARCIAL.....	16
Práctica 16: Ordenamiento Burbuja de Enteros, Caracteres y Nuevo Tipo de Dato.....	16
Práctica 17: Ordenamiento Merge Sort de Enteros, Caracteres y Nuevo Tipo de Dato..	17
Práctica 18: Ordenamiento quick sort.....	18
Práctica 19: Ordenamiento indirecto burbuja.....	19
Práctica 20: Ordenamiento MergeSort Indirecto (POO).....	19
Práctica 21: Ordenamiento QuickSort Indirecto (POO).....	20
Práctica 1: Punteros para el Manejo Indirecto con Clases y Estructuras.....	20
Práctica 2: ADT Colección (Tipo de Dato Abstracto).....	21
Ejercicio 3: Pila estática (arreglo).....	22
Ejercicio 4: Cola estática (arreglo).....	22
Ejercicio 5: Lista estática (arreglo).....	23
Ejercicio 6: Pila dinámica (con nodos).....	24
Práctica 7: Cola.....	25
Práctica 7 Bis: Lista.....	25
Práctica 8: Pila.....	26
Práctica 9: Cola.....	26
Práctica 10: Lista Dinámica.....	27
TERCER PARCIAL.....	28
Práctica 1: Escribir y Leer en XML datos básicos, arreglos, creados arreglos y los ADT28	
Practica 2: Escribir y leer txt datos basicos, arreglos, creados arreglos y los ADT.....	29
Práctica 3: Escribir y Leer JSON (Datos Básicos, Arreglos y ADT POO).....	30
Práctica 4: Archivos.....	30
Práctica 5: Implementación de Grafos, Algoritmo de Dijkstra y Persistencia en XML (Grafo).....	31

Práctica 6: Digrafo POO.....	32
Práctica 7: Árbol con Lectura XML/JSON y Algoritmo de Dijkstra.....	33

INTRODUCCIÓN

La informática y el desarrollo de software se han convertido en herramientas fundamentales para la solución de problemas en prácticamente todas las áreas del conocimiento. Dentro de este contexto, la programación permite diseñar algoritmos capaces de procesar información, automatizar tareas y generar soluciones eficientes mediante el uso de estructuras de datos y técnicas de desarrollo cada vez más avanzadas.

Uno de los conceptos más importantes en el estudio de la programación es la **abstracción de datos**, la cual consiste en representar elementos del mundo real mediante estructuras y modelos que permiten manipular información de forma organizada. Gracias a este principio es posible simplificar problemas complejos, enfocándose únicamente en las características relevantes de los datos y ocultando los detalles innecesarios de su implementación.

Durante el desarrollo de la asignatura **Abstracción y Uso de Datos** se abordaron diferentes temas relacionados con la Programación Estructurada y la Programación Orientada a Objetos (POO). La Programación Estructurada permitió comprender la importancia de utilizar funciones, procedimientos, ciclos y estructuras de control para construir programas claros y eficientes. Por otra parte, la Programación Orientada a Objetos introdujo conceptos como clases, objetos, encapsulamiento, herencia, sobrecarga y sobreescripción de métodos, facilitando la reutilización de código y la creación de aplicaciones más organizadas y escalables.

A lo largo de las prácticas realizadas se trabajó con diferentes tipos de datos, desde variables básicas hasta estructuras y clases creadas por el usuario para representar entidades del mundo real, como automóviles y personas. Asimismo, se implementaron arreglos, matrices y punteros, permitiendo comprender cómo se almacenan y manipulan los datos en memoria, así como la importancia de una correcta administración de los recursos del sistema.

Otro aspecto fundamental del curso fue el estudio de los algoritmos de búsqueda y ordenamiento. Se implementaron métodos como Burbuja, Merge Sort y QuickSort, los cuales permitieron analizar diferentes estrategias para organizar y localizar información dentro de conjuntos de datos. Estos algoritmos son ampliamente utilizados en aplicaciones reales debido a su importancia en el procesamiento eficiente de grandes volúmenes de información.

Además, se trabajó con estructuras dinámicas como pilas, colas y listas enlazadas, así como con Tipos de Datos Abstractos (ADT), herramientas esenciales para el diseño de software moderno. Estas estructuras permiten almacenar información de manera flexible y eficiente, adaptándose a diferentes necesidades según el problema que se desea resolver.

Como parte del proceso de persistencia de información, se desarrollaron programas capaces de almacenar y recuperar datos utilizando diferentes formatos de archivos, entre ellos TXT, CSV, JSON y XML. Estos formatos son ampliamente utilizados en la industria para el intercambio y almacenamiento de información entre sistemas, aplicaciones y bases de datos. Su implementación permitió comprender la importancia de la organización, validación y recuperación de datos dentro de un entorno computacional.

Finalmente, se abordó el estudio de los grafos y digrafos, estructuras matemáticas utilizadas para representar relaciones entre elementos. Mediante la implementación del algoritmo de Dijkstra fue posible calcular rutas más cortas entre nodos, demostrando la aplicación práctica de estas estructuras en sistemas de navegación, redes de comunicación, transporte y optimización de procesos.

El presente reporte tiene como finalidad documentar las prácticas desarrolladas durante los tres parciales de la materia, describiendo los objetivos, dificultades encontradas, errores detectados y soluciones implementadas. Asimismo, busca evidenciar el aprendizaje adquirido en el manejo de estructuras de datos, algoritmos, programación orientada a objetos, persistencia de información y modelado de problemas, conocimientos fundamentales para la formación profesional en el área de la ingeniería en sistemas y el desarrollo de software.

PRIMER PARCIAL

Práctica 1: Hola Mundo

Dificultades y errores

Durante el desarrollo de esta práctica la principal dificultad se presentó al integrar la generación de archivos de registro en los cuatro formatos requeridos (TXT, CSV, JSON y XML) sin alterar la estructura original del programa. La clase HolaMundo únicamente contenía el método holaMundo(), por lo que cualquier modificación interna habría cambiado su funcionamiento original.

Surgió también la pregunta de dónde colocar la lógica de generación de archivos: añadirla directamente dentro de la clase implicaba mezclar responsabilidades distintas, lo cual va en contra de los principios de la Programación Orientada a Objetos. Adicionalmente, fue necesario determinar qué información registrar, ya que el programa únicamente imprime un mensaje en pantalla sin calcular ni procesar datos adicionales.

Corrección

Para solucionar estos problemas se diseñó una librería independiente denominada logger.h, cuya función es centralizar toda la lógica de generación de archivos sin interferir con las clases originales. Esta librería define la clase Logger, la cual expone dos métodos: registrar(), que almacena en memoria cada operación o evento del programa, y guardar(), que escribe los cuatro archivos al finalizar la ejecución.

Gracias a este diseño, la clase HolaMundo se mantuvo completamente intacta. En el main() únicamente se instanció el Logger, se registró la salida del método holaMundo() y se llamó a guardar() antes de terminar el programa. Esto garantizó que los archivos generados reflejaran siempre la última ejecución.

Práctica 2: Suma de Dos Números como Parámetros

Dificultades y errores

En esta práctica la dificultad principal fue registrar las operaciones de suma correctamente, dado que el método sumar() de la clase Calculadora no retorna ningún valor: calcula e imprime el resultado directamente en pantalla. Esto significó que el Logger no podía obtener el resultado desde el valor de retorno del método, sino que debía calcularlo de forma independiente o conocerlo de antemano.

Otro reto consistió en que el programa realiza dos llamadas fijas a sumar() con valores predefinidos (10, 5) y (25, 15), lo cual simplificó el registro, pero también exigió que los valores almacenados en los archivos coincidieran exactamente con lo que el programa imprime en pantalla.

Corrección

La solución consistió en registrar manualmente cada operación después de su llamada correspondiente, pasando al Logger los mismos valores que el método utiliza internamente. Dado que los parámetros y resultados son conocidos, se registraron directamente con log.registrar("suma", "sumar(10, 5)", 15) y log.registrar("suma", "sumar(25, 15)", 40),

asegurando que los archivos generados fueran consistentes con la salida en pantalla. La clase Calculadora original no fue modificada en ningún momento.

Práctica 3: Calculadora Básica

Dificultades y errores

Esta práctica presentó una mayor complejidad debido a que el programa es interactivo: el usuario puede realizar múltiples operaciones en una misma sesión. Esto implicó que el Logger debía acumular dinámicamente todas las operaciones realizadas durante la ejecución, sin perder ningún registro.

Otro punto de dificultad fue el manejo de la división entre cero. El método dividir() de la clase Calculadora no retorna ningún valor y gestiona internamente el error al imprimir un mensaje en pantalla. Fue necesario replicar esa lógica de validación en el main() para registrar correctamente si la operación fue exitosa o si ocurrió un error, sin duplicar el código dentro de la clase original.

También surgieron complicaciones relacionadas con los diferentes tipos de retorno de los métodos: sumar(), restar() y multiplicar() retornan float, mientras que dividir() es void, lo que requirió un tratamiento distinto para cada caso al momento de registrar los eventos.

Corrección

Para resolver estos problemas se instanció el Logger al inicio del main() y se añadió una llamada a log.registrar() dentro de cada case del switch, inmediatamente después de ejecutar la operación correspondiente. Para la división, se realizó la validación de $b \neq 0$ en el main() antes de llamar a dividir(), permitiendo registrar tanto el resultado correcto como el mensaje de error según correspondiera.

Al finalizar el ciclo do-while, se llamó a log.guardar() para escribir los cuatro archivos con el historial completo de la sesión. De esta forma la clase Calculadora permaneció sin modificaciones y los archivos generados reflejaron fielmente todas las operaciones realizadas por el usuario.

Práctica 4: Calculadora con Sobrecarga de Funciones

Dificultades y errores

La dificultad principal en esta práctica fue distinguir y registrar correctamente las tres versiones sobrecargadas del método sumar() de la clase CalculadoraSobrecarga. Cada variante recibe un número diferente de parámetros: ninguno, dos o tres, y el Logger debía identificar cuál fue llamada en cada iteración para registrarla con un tipo descriptivo adecuado.

El método sumar() sin parámetros no retorna ningún valor y opera con valores fijos internos (10 y 20), por lo que su resultado debía conocerse de antemano para registrarlo correctamente. Las variantes con dos y tres parámetros sí retornan float, lo que facilitó su registro pero exigió capturar el valor de retorno antes de mostrarlo en pantalla.

Corrección

La solución consistió en capturar el valor de retorno en una variable local para las versiones con dos y tres parámetros, registrándola inmediatamente con `log.registrar()` antes de continuar. Para la versión sin parámetros, se registró el resultado fijo de 30 (suma de 10 + 20) directamente. Cada case del switch utilizó un tipo descriptivo diferente: `sumar()`, `sumar(a,b)` y `sumar(a,b,c)`, permitiendo identificar claramente en los archivos qué variante fue invocada. La clase `CalculadoraSobrecarga` no fue modificada en ningún momento.

Práctica 5: Herencia.

Dificultades y errores.

La dificultad principal en esta práctica fue crear la clase **CalculadoraNueva** utilizando como base la clase **CalculadoraSobrecarga** sin modificar el funcionamiento original de la clase padre. Fue necesario comprender correctamente el concepto de herencia para reutilizar los atributos y métodos ya existentes, evitando duplicar código innecesariamente.

Otro inconveniente fue asegurar que los nuevos métodos agregados en la clase derivada coexistieran correctamente con los métodos heredados. Durante las pruebas iniciales se presentaron errores de acceso y confusión entre los métodos propios de la clase hija y los heredados de la clase padre.

Corrección.

La solución consistió en declarar la clase **CalculadoraNueva** como derivada pública de **CalculadoraSobrecarga**, permitiendo el acceso a todos los métodos públicos de la clase base. Posteriormente se agregaron los nuevos métodos requeridos manteniendo intacta la estructura original del programa.

De esta manera se logró reutilizar correctamente el código existente y comprobar que los métodos heredados podían utilizarse junto con las nuevas funciones implementadas en la clase derivada sin generar conflictos.

Práctica 6: Herencia y Sobreescritura.

Dificultades y errores.

La principal dificultad de esta práctica fue implementar nuevos métodos matemáticos utilizando únicamente operaciones básicas. La multiplicación debía realizarse mediante sumas sucesivas, la división mediante restas sucesivas y la potencia mediante multiplicaciones sucesivas, lo que obligó a desarrollar algoritmos alternativos al uso de operadores tradicionales.

Además, fue necesario sobrescribir el método que recibe dos parámetros para modificar su comportamiento sin afectar los demás métodos heredados. Durante las primeras pruebas se obtuvieron resultados incorrectos debido a errores en los ciclos de repetición utilizados para las operaciones sucesivas.

Corrección.

La solución consistió en redefinir los métodos necesarios dentro de la clase derivada y utilizar estructuras repetitivas para ejecutar cada operación paso a paso. La multiplicación se implementó acumulando sumas repetidas, la división restando el divisor hasta alcanzar un valor menor que éste y la potencia mediante multiplicaciones consecutivas.

Con estas modificaciones se logró mantener la funcionalidad heredada mientras se personalizaba el comportamiento requerido para la nueva calculadora, demostrando correctamente el uso de herencia y sobrescritura.

Práctica 7: Creación de Tipo de Dato (Auto-Persona)

Dificultades y errores.

La principal dificultad consistió en crear un nuevo tipo de dato utilizando programación estructurada y programación orientada a objetos. Fue necesario identificar correctamente los atributos que describen un automóvil, en este caso el precio y el año de fabricación.

Durante las pruebas iniciales se presentaron errores de captura y visualización de los datos debido a una incorrecta asignación de valores en las variables utilizadas para almacenar la información ingresada por el usuario.

La dificultad principal fue representar la información completa de una persona mediante una estructura de datos adecuada. Se requería almacenar varios atributos como nombre, apellido paterno, apellido materno, género y edad, manteniendo una organización clara de los datos.

Durante las pruebas surgieron problemas relacionados con la captura de cadenas de texto, especialmente cuando se intentaban ingresar espacios en los nombres o apellidos, ocasionando lecturas incompletas de la información.

Corrección.

La solución consistió en definir adecuadamente los atributos del automóvil y crear métodos específicos para capturar y mostrar la información. Una vez corregida la captura de datos, el programa pudo almacenar correctamente el precio y el año del vehículo.

Con ello se obtuvo un nuevo tipo de dato funcional que permitió representar objetos del mundo real mediante estructuras y clases, facilitando la comprensión de los conceptos fundamentales de la programación orientada a objetos.

La solución consistió en definir correctamente cada atributo de la persona y utilizar métodos de captura apropiados para almacenar cadenas de texto completas. Posteriormente se implementó un método de visualización que mostrara toda la información registrada de manera ordenada.

Gracias a estas correcciones se logró representar adecuadamente a una persona mediante un tipo de dato personalizado tanto en programación estructurada como en programación orientada a objetos.

Práctica 7 Bis: Identificar Tamaños.

Dificultades y errores.

La principal dificultad de esta práctica fue identificar correctamente el tamaño en memoria de los diferentes tipos de datos básicos y de los tipos de datos creados por el usuario. Los resultados obtenidos podían variar dependiendo del compilador y la arquitectura del sistema operativo utilizado.

Además, existía confusión sobre la diferencia entre el tamaño de una variable individual y el tamaño total de una estructura o clase que contiene varios atributos.

Corrección.

La solución consistió en utilizar el operador **sizeof()** para obtener de manera precisa la cantidad de memoria ocupada por cada tipo de dato. Se realizaron pruebas tanto con tipos básicos como enteros, flotantes y caracteres, así como con las estructuras y clases creadas previamente.

Esto permitió comparar los tamaños obtenidos y comprender mejor la forma en que los datos son almacenados en memoria dentro de un programa.

Práctica 8: Promedio.

Dificultades y errores.

La principal dificultad fue procesar correctamente un conjunto de cinco números para obtener diversas estadísticas. Fue necesario calcular el promedio, la suma, identificar el valor máximo y mínimo y determinar la media de los datos ingresados.

Durante las primeras pruebas aparecieron errores en la inicialización de las variables utilizadas para los cálculos, provocando resultados incorrectos especialmente en la búsqueda del máximo y mínimo.

Corrección.

La solución consistió en almacenar los cinco valores en una estructura adecuada y recorrerlos mediante ciclos para realizar los cálculos necesarios. El promedio se obtuvo mediante la suma total dividida entre la cantidad de elementos, mientras que los valores máximo y mínimo se determinaron mediante comparaciones sucesivas.

Una vez corregidas las inicializaciones y verificadas las operaciones matemáticas, el programa generó correctamente todas las estadísticas solicitadas.

Práctica 9: Promedio con arreglo

Dificultades y errores

Durante el desarrollo de este programa se presentaron algunas dificultades relacionadas con la generación de archivos en distintos formatos. Inicialmente, el programa solamente

calculaba la suma, promedio, valor máximo y valor mínimo de los datos almacenados en el arreglo. Sin embargo, posteriormente fue necesario exportar estos resultados a archivos TXT, CSV, JSON y XML.

La principal dificultad fue comprender las diferencias entre cada formato de almacenamiento. En el caso de TXT, la información podía escribirse de forma simple; sin embargo, para CSV fue necesario separar correctamente los datos mediante comas para que pudieran ser interpretados por programas como Excel. En JSON surgieron problemas relacionados con la sintaxis, especialmente en el uso correcto de llaves, comillas y comas. Por otro lado, XML requirió el uso adecuado de etiquetas de apertura y cierre para mantener una estructura válida.

También fue necesario evitar la duplicación de código, ya que cada formato requería acceder a los mismos resultados obtenidos mediante los métodos de cálculo.

Corrección

Para solucionar estos problemas se diseñaron métodos independientes para cada formato de archivo: `generarTXT()`, `generarCSV()`, `generarJSON()` y `generarXML()`. Cada método se encargó exclusivamente de la generación de un formato específico, mejorando la organización y mantenimiento del programa.

Además, se creó el método `generarArchivos()`, cuya función es centralizar la generación de todos los archivos desde un solo punto. Esto permitió reutilizar los métodos de cálculo ya existentes y mantener una estructura orientada a objetos más ordenada y eficiente.

Práctica 10: Promedio con puntero al arreglo

Dificultades y errores

En este ejercicio la principal complejidad fue el uso de punteros para acceder a los elementos del arreglo. A diferencia del ejercicio anterior, los datos no se manipulaban directamente mediante índices tradicionales, sino utilizando aritmética de punteros a través de expresiones como `(ptr+i)`.

Durante la implementación se presentaron errores al momento de recorrer el arreglo utilizando el puntero, ya que cualquier error en la referencia podía provocar resultados incorrectos en los cálculos. También se identificó la necesidad de evitar cálculos repetitivos al generar los archivos TXT, CSV, JSON y XML, debido a que todos los formatos requerían exactamente la misma información.

Otra dificultad consistió en adaptar los resultados calculados para que pudieran almacenarse correctamente en cada formato, manteniendo la consistencia entre los archivos generados.

Corrección

La solución consistió en inicializar correctamente el puntero mediante el constructor de la clase, asegurando que apuntara al inicio del arreglo. Posteriormente, todos los cálculos

fueron realizados dentro del método `calcular()`, obteniendo una única vez la suma, promedio, máximo y mínimo.

Una vez calculados los resultados, estos fueron enviados como parámetros a los métodos encargados de generar los archivos. De esta forma se evitó recalcular la información varias veces y se logró una mejor optimización del programa. Finalmente, se verificó que la información exportada en TXT, CSV, JSON y XML fuera consistente y representara correctamente los resultados obtenidos.

Práctica 11: Matrices

Dificultades y errores

Este ejercicio presentó un mayor nivel de complejidad debido al manejo simultáneo de varias matrices y operaciones matemáticas. Además de capturar las matrices A y B, fue necesario almacenar la matriz resultante de la multiplicación por una constante y la matriz producto de A por B.

La generación de archivos representó un reto importante, ya que la información ya no consistía en valores individuales sino en estructuras bidimensionales de 3x3. En el archivo CSV fue necesario diseñar una representación adecuada para almacenar filas completas dentro de una misma línea. En JSON surgieron dificultades relacionadas con la construcción de arreglos anidados para representar correctamente las matrices. Por su parte, XML requirió una organización jerárquica mediante etiquetas específicas para cada matriz y cada fila.

También fue necesario asegurar que las matrices de resultado estuvieran completamente calculadas antes de generar los archivos de salida, ya que de lo contrario podrían almacenarse datos incorrectos o incompletos.

Corrección

Se implementaron métodos especializados para cada formato de archivo, permitiendo exportar la información de manera organizada. En el caso de JSON se utilizaron arreglos anidados para representar cada fila de las matrices, mientras que en XML se emplearon etiquetas específicas para conservar la estructura jerárquica de los datos.

Asimismo, se estableció una secuencia correcta de ejecución: primero la captura de las matrices, después la multiplicación por la constante, posteriormente la multiplicación entre matrices y finalmente la generación de los archivos. Esto garantizó que todos los resultados estuvieran disponibles antes de ser exportados.

Finalmente, se realizaron pruebas verificando que los valores almacenados en TXT, CSV, JSON y XML coincidieran exactamente con los resultados mostrados en pantalla.

Práctica 12: Nuevo tipo de dato como arreglo

Dificultades y errores

Durante el desarrollo de esta práctica se presentaron dificultades al trabajar con arreglos de objetos, ya que fue necesario almacenar y administrar información de dos clases diferentes (Auto y Persona). A diferencia de los arreglos tradicionales de números, en este caso cada posición del arreglo contenía varios atributos y métodos asociados al objeto.

También surgieron complicaciones al momento de generar los archivos TXT, CSV, JSON y XML, debido a que era necesario exportar información de distintos tipos de datos, incluyendo cadenas de texto, números enteros y registros completos de autos y personas. Cada formato requería una estructura diferente para representar correctamente la información almacenada en los arreglos.

En el caso de JSON fue necesario organizar los datos mediante objetos y arreglos, cuidando el uso correcto de llaves, comillas y comas. Para XML se requirió construir una estructura jerárquica mediante etiquetas que representaran adecuadamente cada auto y cada persona. Asimismo, fue necesario verificar que la información capturada por el usuario se almacenara correctamente antes de generar los archivos de salida.

Corrección

Para solucionar estos problemas se utilizaron arreglos de objetos de las clases Auto y Persona, permitiendo almacenar múltiples registros de forma organizada. Se implementaron métodos de captura y visualización dentro de cada clase para facilitar el manejo de la información y mantener una estructura orientada a objetos.

Además, se desarrollaron funciones independientes para generar los archivos TXT, CSV, JSON y XML. Cada función se encargó de recorrer los arreglos de autos y personas, obteniendo la información necesaria para construir correctamente cada formato de salida.

Finalmente, se realizaron pruebas verificando que los datos capturados coincidieran con la información almacenada en los archivos generados, garantizando la integridad de los registros y el correcto funcionamiento del programa.

Práctica 13: Nuevo tipo de dato con puntero al arreglo

Dificultades y errores

Durante el desarrollo del programa se presentó la necesidad de almacenar la información capturada por el usuario en diferentes formatos de archivo. Inicialmente, el programa únicamente permitía registrar y mostrar los datos de automóviles y personas en pantalla, por lo que no existía ningún mecanismo para conservar la información después de finalizar la ejecución. Además, fue necesario analizar en qué parte del programa debía colocarse la generación de archivos para evitar afectar el funcionamiento normal del menú y de las operaciones de captura.

Otro inconveniente surgió al agregar nuevos atributos a las clases, ya que fue necesario modificar tanto los métodos de captura como los de visualización y almacenamiento. Esto provocó algunos errores relacionados con la lectura de datos y la actualización de la información mostrada en pantalla.

Corrección

Para solucionar estos problemas se incorporó la biblioteca <fstream>, la cual permitió crear y escribir archivos de texto y otros formatos. Se implementó un mecanismo de actualización automática que genera nuevamente los archivos TXT, CSV, XML, HTML y JSON cada vez que se registra un nuevo automóvil o una nueva persona.

Asimismo, se verificó que todos los atributos capturados se almacenaran correctamente en los archivos generados. Finalmente, se realizaron pruebas de funcionamiento para comprobar que la información mostrada en pantalla coincidiera con la información exportada.

Práctica 14: Recursividad

Dificultades y errores

La principal dificultad consistió en que el programa estaba diseñado únicamente para mostrar los resultados de las operaciones recursivas en la consola. No existía una estructura que permitiera conservar la información de la última operación realizada ni generar archivos con los resultados obtenidos.

También fue necesario identificar qué datos debían exportarse, ya que el programa realiza diferentes tipos de operaciones matemáticas, como multiplicación, división, potencia, factorial y sucesión de Fibonacci. Cada operación produce resultados distintos, por lo que fue necesario diseñar una forma uniforme de almacenamiento.

Corrección

Se incorporaron variables auxiliares para almacenar el nombre de la operación ejecutada y el resultado obtenido. Posteriormente, se desarrolló un procedimiento encargado de generar automáticamente archivos en los formatos solicitados después de cada cálculo realizado por el usuario.

Con estas modificaciones fue posible conservar los resultados de las operaciones recursivas y mantener actualizada la información en todos los archivos generados. Además, se verificó que los resultados almacenados coincidieran con los mostrados en pantalla.

Practica 15: Fractales Recursivos (Polvo de Cantor y Triángulo de Sierpinski)

Problema

Desarrollar un programa capaz de generar fractales mediante el uso de recursividad. El sistema debía permitir al usuario seleccionar entre la generación del Polvo de Cantor o el Triángulo de Sierpinski, mostrando visualmente la construcción de cada fractal en la consola. El objetivo principal fue aplicar los conceptos de recursividad para resolver problemas de geometría fractal.

Desarrollo y solución

Se diseñó una clase llamada `GeneradorFractales`, encargada de generar y mostrar los fractales seleccionados por el usuario. Para el Polvo de Cantor se implementó una función recursiva que divide una línea en tres partes iguales y elimina visualmente el segmento central en cada nivel de recursión. El proceso continúa hasta alcanzar el nivel indicado por el usuario.

Para el Triángulo de Sierpinski se utilizó una matriz bidimensional que representa un tablero de caracteres. Mediante llamadas recursivas, el algoritmo divide el triángulo original en tres subtriángulos más pequeños hasta llegar al caso base, donde se coloca un carácter de asterisco para representar cada punto del fractal.

El programa incorpora un menú interactivo que permite seleccionar el tipo de fractal a generar, facilitando la ejecución de ambos algoritmos dentro de una misma aplicación.

Dificultades y errores

Durante el desarrollo surgieron dificultades relacionadas con la implementación de la recursividad, principalmente en la definición correcta de los casos base para evitar ciclos infinitos y errores de ejecución. También fue necesario verificar cuidadosamente las coordenadas utilizadas para construir el Triángulo de Sierpinski, ya que pequeños errores en los cálculos provocaban desplazamientos incorrectos o figuras incompletas.

En el caso del Polvo de Cantor, se presentaron problemas al calcular el tamaño de los segmentos en cada nivel de división, especialmente cuando la longitud de la línea disminuía considerablemente. Asimismo, fue necesario controlar adecuadamente los espacios en blanco para representar correctamente la eliminación del segmento central.

Corrección

Para solucionar estos inconvenientes se establecieron condiciones base claras en cada función recursiva, garantizando la finalización correcta de los procesos. Se realizaron múltiples pruebas modificando los niveles de recursión para verificar que los fractales conservaran su forma característica en cada iteración.

Además, se ajustaron las coordenadas utilizadas en el Triángulo de Sierpinski y se validó el cálculo de los segmentos en el Polvo de Cantor, asegurando una representación gráfica correcta. Finalmente, se comprobó el funcionamiento del menú interactivo y la generación adecuada de ambos fractales, obteniendo resultados visuales consistentes y acordes con la teoría de los fractales recursivos.

SEGUNDO PARCIAL

Práctica 16: Ordenamiento Burbuja de Enteros, Caracteres y Nuevo Tipo de Dato

Problema

Implementar el algoritmo de ordenamiento Burbuja para ordenar conjuntos de datos de tipo entero, caracteres y un nuevo tipo de dato definido mediante Programación Orientada a Objetos. Además, generar archivos TXT, CSV, JSON y XML con los resultados obtenidos.

Desarrollo y solución

Se desarrolló una clase Burbuja encargada de realizar el ordenamiento de enteros, caracteres y objetos de tipo Persona. El algoritmo Burbuja compara elementos adyacentes e intercambia sus posiciones cuando se encuentran en un orden incorrecto, repitiendo este proceso hasta ordenar completamente los datos.

El programa cuenta con un menú interactivo que permite capturar los diferentes tipos de datos y mostrarlos una vez ordenados. Para el nuevo tipo de dato se utilizó una estructura Persona compuesta por los atributos edad e inicial, realizando el ordenamiento con base en la edad.

Después de finalizar el ordenamiento, el programa genera automáticamente archivos TXT, CSV, JSON y XML que almacenan los resultados ordenados, permitiendo conservar y consultar la información en distintos formatos.

Dificultades y errores

Una de las principales dificultades fue adaptar el algoritmo Burbuja para trabajar con distintos tipos de datos, especialmente con el nuevo tipo de dato Persona, ya que fue necesario comparar el atributo edad para determinar el orden correcto.

También surgieron complicaciones al generar los archivos TXT, CSV, JSON y XML, debido a que cada formato requiere una estructura diferente para almacenar la información. Fue necesario asegurar que los datos ordenados se escribieran correctamente en cada archivo sin perder información.

Corrección

Para resolver estos problemas se implementaron funciones específicas de ordenamiento para enteros, caracteres y personas. Asimismo, se desarrollaron funciones independientes para generar los archivos TXT, CSV, JSON y XML, permitiendo almacenar la información de manera adecuada según el formato correspondiente.

Finalmente, se realizaron pruebas con distintos conjuntos de datos para verificar que el algoritmo Burbuja ordenara correctamente los elementos y que los archivos generados coincidieran con los resultados mostrados en pantalla, garantizando el correcto funcionamiento del programa.

Práctica 17: Ordenamiento Merge Sort de Enteros, Caracteres y Nuevo Tipo de Dato

Problema

Implementar el algoritmo de ordenamiento Merge Sort para ordenar conjuntos de datos de tipo entero, caracteres y un nuevo tipo de dato definido mediante Programación Orientada a Objetos. Además, generar archivos TXT, CSV, JSON y XML con la información ordenada.

Desarrollo y solución

Se desarrolló una clase MergeSort encargada de realizar el ordenamiento de enteros, caracteres y objetos de tipo Persona. El algoritmo divide los datos en subconjuntos más pequeños y posteriormente los combina de manera ordenada mediante el proceso de mezcla (*merge*).

El programa cuenta con un menú interactivo que permite capturar los datos, ejecutar el ordenamiento y mostrar los resultados en pantalla. Una vez ordenados, los datos se almacenan automáticamente en archivos TXT, CSV, JSON y XML para facilitar su consulta y reutilización.

Dificultades y errores

Una de las principales dificultades fue implementar correctamente el proceso de división y mezcla de los arreglos, ya que fue necesario utilizar arreglos temporales para almacenar los subconjuntos durante el ordenamiento. También surgieron complicaciones al adaptar el algoritmo para trabajar con diferentes tipos de datos, especialmente con el nuevo tipo de dato Persona, donde fue necesario realizar las comparaciones utilizando el atributo edad.

Además, se presentaron dificultades al generar los archivos TXT, CSV, JSON y XML, ya que cada formato requiere una estructura diferente para almacenar correctamente la información ordenada.

Corrección

Para solucionar estos problemas se implementaron funciones de mezcla específicas para enteros, caracteres y personas, asegurando que las comparaciones se realizaran correctamente en cada caso. Asimismo, se desarrollaron funciones independientes para generar los archivos TXT, CSV, JSON y XML, verificando que la información almacenada coincidiera con los datos mostrados por el programa.

Finalmente, se realizaron pruebas con distintos conjuntos de datos para comprobar el correcto funcionamiento del algoritmo de ordenamiento y la generación adecuada de todos los archivos de salida.

Práctica 18: Ordenamiento quick sort

Dificultades y errores

Durante el desarrollo de esta práctica se presentaron dificultades al implementar el algoritmo QuickSort para tres tipos distintos de datos: enteros, caracteres y estructuras Persona. Aunque el algoritmo es el mismo en esencia, fue necesario adaptar las comparaciones y el proceso de partición para cada tipo de dato, especialmente en el caso de las estructuras, donde el criterio de ordenamiento fue la edad de cada persona.

Otra dificultad importante fue el manejo de memoria dinámica mediante el uso de punteros y la instrucción new para crear arreglos de tamaño variable. Fue necesario controlar correctamente la reserva y liberación de memoria utilizando delete[] para evitar pérdidas de memoria durante la ejecución del programa.

En la sección de personas surgieron complicaciones al ordenar los registros, ya que no solo debía ordenarse un valor numérico, sino mantener asociada la inicial correspondiente a cada persona. Durante las pruebas se verificó que los intercambios realizados por QuickSort conservaran correctamente toda la información del registro.

También se presentaron dificultades en la generación de archivos TXT, CSV, JSON y XML. Debido a que el programa permite trabajar con enteros, caracteres y personas, fue necesario desarrollar funciones independientes para cada tipo de dato: generarInt(), generarChar() y generarPersona(). Cada una debía construir correctamente la estructura correspondiente según el formato solicitado.

En el caso de JSON se requirió construir arreglos y objetos utilizando correctamente llaves, corchetes, comillas y comas. Para XML fue necesario definir etiquetas específicas como < y >, así como etiquetas internas para representar números, caracteres y registros de personas. Además, se verificó que la información almacenada en los archivos coincidiera exactamente con los datos ordenados mostrados en pantalla.

Corrección

Para solucionar estos problemas se implementaron funciones específicas de QuickSort para enteros, caracteres y estructuras Persona. Cada función utiliza un proceso de partición adecuado al tipo de dato que se está ordenando, permitiendo reutilizar el algoritmo manteniendo su eficiencia.

Se utilizaron arreglos dinámicos creados con new para permitir que el usuario definiera la cantidad de elementos a ordenar, liberando posteriormente la memoria con delete[] para garantizar un manejo adecuado de los recursos del sistema.

Asimismo, se desarrollaron las funciones generarInt(), generarChar() y generarPersona(), encargadas de crear automáticamente los archivos TXT, CSV, JSON y XML después de realizar el ordenamiento. Estas funciones recorren los arreglos ya ordenados y construyen la estructura correspondiente para cada formato de almacenamiento.

Finalmente, se realizaron diversas pruebas con números enteros, caracteres y registros de personas para verificar que el algoritmo QuickSort ordenara correctamente los datos y que

la información generada en los archivos TXT, CSV, JSON y XML coincidiera con los resultados mostrados por el programa.

Práctica 19: Ordenamiento indirecto burbuja

Dificultades y errores

Durante el desarrollo de esta práctica se presentaron dificultades al implementar el método de ordenamiento indirecto Burbuja para enteros, caracteres y estructuras Persona. La principal diferencia respecto a un ordenamiento tradicional fue que no se intercambiaron directamente los datos, sino los índices almacenados en el arreglo auxiliar idx.

También surgieron complicaciones al trabajar con arreglos dinámicos creados mediante new, ya que el tamaño dependía de la cantidad de elementos ingresados por el usuario. Además, fue necesario verificar que las estructuras Persona se ordenaran correctamente utilizando la edad como criterio de comparación.

Otra dificultad fue la generación de los archivos TXT, CSV, JSON y XML, ya que los datos debían guardarse respetando el orden establecido por el arreglo de índices y no por la posición original de los elementos.

Corrección

Para resolver estos problemas se implementó un ordenamiento indirecto Burbuja utilizando el arreglo idx, el cual almacena las posiciones de los datos ordenados sin modificar el arreglo original.

Asimismo, se utilizaron funciones específicas para ordenar enteros, caracteres y personas, tomando la edad como criterio de ordenamiento para las estructuras Persona.

Finalmente, se desarrollaron las funciones generarInt(), generarChar() y generarPersona(), encargadas de crear automáticamente los archivos TXT, CSV, JSON y XML con los datos ya ordenados, verificando que la información almacenada coincidiera con la mostrada en pantalla.

Práctica 20: Ordenamiento MergeSort Indirecto (POO)

Dificultades y errores

Durante el desarrollo de esta práctica se presentaron dificultades al registrar los resultados de un algoritmo de ordenamiento indirecto que opera sobre arreglos de punteros. El algoritmo MergeSort reordena los punteros en lugar de los datos originales, por lo que fue necesario recorrer el arreglo de punteros después de ordenar para construir la representación del resultado.

Otra dificultad fue el manejo de tres tipos de datos distintos (enteros, caracteres y objetos PersonaPOO) dentro del mismo programa. Cada tipo requería una función separada para construir el string de resultado que se almacena en el Logger, ya que los objetos PersonaPOO no pueden convertirse a string directamente mediante to_string().

También fue necesario asegurar que los arreglos dinámicos de datos y punteros se liberaran correctamente con `delete[]` después de generar el registro, para evitar fugas de memoria durante la sesión.

Corrección

La solución consistió en construir el string de resultado dentro de cada función (`ordenarEnteros()`, `ordenarCaracteres()` y `ordenarPersonasPOO()`) utilizando un `ostringstream`, recorriendo el arreglo de punteros una vez concluido el ordenamiento. Para los objetos `PersonaPOO`, se aprovechó el operador `<<` sobrecargado escribiendo cada objeto en un `ostringstream` temporal para obtener su representación textual. El `Logger` se pasó por referencia a cada función y se llamó a `log.guardar()` al finalizar el ciclo del menú, garantizando que los archivos reflejaran todas las operaciones de la sesión.

Práctica 21: Ordenamiento QuickSort Indirecto (POO)

Dificultades y errores

Esta práctica presentó dificultades similares a la del `MergeSort`, con la particularidad de que el algoritmo `QuickSort` utiliza una función de partición que intercambia punteros en función de un pivote. Al operar de forma recursiva, el resultado final del ordenamiento no está disponible en un único punto del código, sino que emerge tras la recursión completa.

Al igual que en el `MergeSort`, fue necesario manejar tres tipos de datos distintos y construir representaciones textuales del resultado para cada uno. Para los objetos `PersonaPOO`, el operador de comparación utilizado en el `QuickSort` es `operator<()` (menor que), a diferencia del `MergeSort` que utiliza `operator<=()`, lo que exigió verificar que los operadores estuvieran correctamente implementados en la clase para evitar comportamientos inesperados durante el ordenamiento.

Corrección

La corrección siguió el mismo patrón utilizado en el `MergeSort`: construir el string de resultado con un `ostringstream` después de completar el ordenamiento en cada función auxiliar, registrar el evento en el `Logger` y liberar la memoria dinámica antes de retornar. Se verificó que el operador `operator<()` de la clase `PersonaPOO` comparara correctamente las edades, y se realizaron pruebas con los tres tipos de datos para confirmar que el `QuickSort` producía resultados consistentes con los mostrados en pantalla. Los archivos generados reflejaron fielmente el resultado de la última ejecución del programa.

Práctica 1: Punteros para el Manejo Indirecto con Clases y Estructuras

Dificultades y errores

Durante el desarrollo de esta práctica se presentaron dificultades al integrar el `Logger` en un programa que maneja memoria dinámica mediante punteros. La lista enlazada se construye nodo a nodo en tiempo de ejecución, por lo que el número de operaciones a registrar no es fijo: depende completamente de las acciones del usuario durante la sesión.

Otra dificultad surgió al registrar la opción de mostrar la cadena. La función `mostrarNodos()` recorre la lista e imprime cada nodo en pantalla, pero no retorna ningún valor. Para generar un registro útil en los archivos fue necesario recorrer la lista nuevamente en el `main()` para construir una representación textual del estado actual de la cadena.

También fue necesario asegurar que `log.guardar()` se llamara después de `liberarMemoria()` y antes del fin del programa, para que los archivos reflejaran correctamente todas las operaciones de la sesión, incluyendo la salida.

Corrección

Para resolver estos problemas se pasó el `Logger` por referencia a las funciones `capturaNodo()` e `impresion()`, permitiendo registrar cada evento desde dentro de las mismas funciones auxiliares sin modificar la clase `Nodo` ni las funciones de la librería. En `impresion()` se construyó un string recorriendo la lista con un puntero temporal, concatenando los valores de cada nodo para generar el snapshot que se almacena en los archivos. Finalmente, `log.guardar()` se colocó al final del `main()` tras la liberación de memoria, garantizando que los cuatro archivos reflejaran el historial completo de la sesión.

Práctica 2: ADT Colección (Tipo de Dato Abstracto)

Dificultades y errores

En esta práctica la principal dificultad fue integrar el `Logger` en un programa que utiliza una jerarquía de clases con herencia y polimorfismo. La colección se maneja a través de un puntero a la clase abstracta `ADT_Coleccion<int>`, lo que significó que el `Logger` debía interactuar con la interfaz pública sin necesidad de conocer la implementación concreta de `Coleccion Arreglo`.

Otro reto fue registrar correctamente el intento de eliminar un elemento cuando la colección está vacía. El método `eliminar()` de `ColeccionArreglo` gestiona internamente este caso imprimiendo un mensaje de error, pero no retorna ningún indicador de éxito o fallo. Fue necesario consultar el estado de la colección mediante `estaVacia()` antes de llamar a `eliminar()` para determinar qué registrar.

Adicionalmente, la redimensión automática del arreglo interno ocurre de forma transparente al insertar elementos cuando se alcanza la capacidad máxima. Aunque este evento se muestra en pantalla, no era necesario registrarlo explícitamente en el `Logger` ya que corresponde a un detalle interno de la implementación.

Corrección

La solución consistió en pasar el `Logger` por referencia a cada función auxiliar (`insertarElemento()`, `mostrarColeccion()` y `eliminarElemento()`). Para el registro de la eliminación, se verificó el estado de la colección con `estaVacia()` antes de llamar a `eliminar()`, registrando el evento como exitoso o como error según correspondiera. El `Logger` se instanció al inicio del `main()` y `log.guardar()` se llamó al finalizar el ciclo, garantizando que los archivos reflejaran el historial completo de la sesión sin modificar ninguna de las clases originales.

Ejercicio 3: Pila estática (arreglo)

Problema:

Implementar una pila utilizando un arreglo de tamaño fijo, respetando el comportamiento LIFO (Last In, First Out), además de permitir almacenar la información generada en archivos externos.

Desarrollo y solución:

Se implementó una pila utilizando un arreglo de tamaño limitado y una variable que representa la posición del tope. Cada vez que se insertaba un elemento mediante la operación Push, el valor del tope se incrementó y el dato se almacenaba en dicha posición. Para eliminar elementos mediante la operación Pop, el valor del tope se decrementa, simulando la salida del último elemento ingresado.

Posteriormente se añadió la funcionalidad de exportación de datos mediante la biblioteca <fstream>. Se creó una función encargada de generar automáticamente archivos en formato TXT, CSV, XML, HTML y JSON. Cada vez que el usuario realizaba una inserción o eliminación, los archivos se actualizaban reflejando el estado actual de la pila.

Dificultades y errores:

Se presentaron errores al intentar insertar elementos cuando la pila ya estaba llena o al intentar eliminar elementos cuando estaba vacía. También surgieron problemas al estructurar correctamente la información para cada formato de archivo y mantener los datos sincronizados después de cada modificación.

Corrección:

Se añadieron validaciones para verificar el estado de la pila antes de cada operación. Asimismo, se implementó una función de exportación automática que se ejecuta después de cada cambio realizado en la estructura, garantizando que los archivos TXT, CSV, XML, HTML y JSON contengan siempre información actualizada y consistente.

Ejercicio 4: Cola estática (arreglo)

Problema:

Implementar una cola utilizando un arreglo, respetando el comportamiento FIFO (First In, First Out), y permitir el almacenamiento automático de los datos en archivos externos.

Desarrollo y solución:

Se desarrolló una cola utilizando un arreglo y dos variables para controlar el inicio y el final de la estructura. Los elementos se insertan al final de la cola y se eliminan desde el frente, manteniendo el orden correcto de procesamiento.

Adicionalmente, se incorporó una función de exportación de datos utilizando la biblioteca <fstream>. Esta función genera automáticamente archivos TXT, CSV, XML, HTML y JSON que almacenan el contenido actual de la cola cada vez que se realiza una operación de inserción o eliminación.

Dificultades y errores:

Durante el desarrollo surgieron problemas relacionados con el control de los límites del arreglo, especialmente cuando la cola se encontraba llena o vacía. También se presentaron dificultades para actualizar correctamente los archivos después de cada modificación realizada por el usuario.

Corrección:

Se implementaron validaciones para impedir operaciones inválidas y evitar errores de ejecución. Asimismo, se verificó que la información exportada coincidiera con el contenido real de la cola y que los archivos se actualizaran automáticamente después de cada operación.

Ejercicio 5: Lista estática (arreglo)

Problema:

Implementar una lista utilizando un arreglo que permita almacenar varios elementos y conservar la información capturada en distintos formatos de archivo.

Desarrollo y solución:

Se utilizó un arreglo de tamaño fijo junto con una variable que controla la cantidad de elementos almacenados. El programa permite insertar elementos y mostrarlos en pantalla mediante un menú interactivo.

Como mejora adicional, se desarrolló una función para generar automáticamente archivos TXT, CSV, XML, HTML y JSON. Cada vez que se agrega un nuevo elemento, los archivos se actualizan con el contenido completo de la lista, permitiendo conservar la información aun después de cerrar el programa.

Dificultades y errores:

Uno de los principales problemas consistió en controlar la capacidad máxima del arreglo para evitar desbordamientos de memoria. También se presentaron dificultades al diseñar la estructura de los diferentes formatos de archivo y al mantenerlos actualizados después de cada inserción.

Corrección:

Se añadieron validaciones para impedir la inserción de elementos cuando la lista estuviera llena. Además, se implementó una función de exportación automática que genera correctamente los archivos solicitados y mantiene la información sincronizada con el contenido de la lista.

Ejercicio 6: Pila dinámica (con nodos)

Problema:

Implementar una pila utilizando memoria dinámica mediante nodos enlazados y almacenar automáticamente la información en diferentes formatos de archivo.

Desarrollo y solución:

Se diseñó una estructura tipo nodo compuesta por un dato y un apuntador al siguiente nodo. La pila se administró mediante un apuntador denominado "tope", el cual siempre señala al último elemento insertado. Cada vez que se realiza una operación Push se crea un nuevo nodo dinámico, mientras que la operación Pop elimina el nodo ubicado en la cima de la pila.

Además, se implementó una función de exportación utilizando la biblioteca <fstream>, encargada de generar archivos TXT, CSV, XML, HTML y JSON. Para ello fue necesario recorrer toda la estructura mediante apuntadores auxiliares y almacenar cada uno de los elementos contenidos en la pila.

Dificultades y errores:

Se presentaron dificultades al trabajar con memoria dinámica y apuntadores, especialmente durante el recorrido de la estructura para exportar los datos. También se identificaron posibles errores relacionados con la liberación incorrecta de memoria y la actualización de los archivos después de cada modificación.

Corrección:

Se corrigió la manipulación de apuntadores mediante recorridos controlados utilizando variables auxiliares. Asimismo, se utilizó correctamente la instrucción delete para liberar memoria cuando un nodo era eliminado. Finalmente, se implementó la actualización automática de los archivos TXT, CSV, XML, HTML y JSON, garantizando que siempre reflejaran el estado actual de la pila dinámica.

Práctica 7: Cola.

Dificultades y errores.

La principal dificultad fue implementar una cola dinámica utilizando punteros, respetando el principio FIFO (First In, First Out). Fue necesario administrar correctamente la memoria dinámica para insertar y eliminar elementos sin perder referencias.

Durante las pruebas iniciales se presentaron errores relacionados con la actualización de los punteros de inicio y final de la cola, ocasionando pérdidas de información y comportamientos inesperados.

Corrección.

La solución consistió en actualizar cuidadosamente los enlaces entre nodos cada vez que se realizaba una inserción o eliminación. También se verificó la correcta liberación de memoria para evitar fugas durante la ejecución.

Con estas correcciones la cola pudo operar de forma estable, permitiendo agregar y retirar elementos en el orden correcto.

Práctica 7 Bis: Lista

Dificultades y errores.

La dificultad principal en esta práctica fue registrar el estado de la lista al momento de mostrarla, dado que el método `mostrar()` de `ListaDinamica` únicamente imprime los nodos en pantalla y no retorna ningún valor. Para generar un registro significativo en los archivos fue necesario disponer de un mecanismo adicional que permitiera obtener el contenido de la lista en forma de texto.

También surgió la necesidad de detectar cuando el usuario intenta eliminar un elemento de una lista vacía. El método `eliminar()` maneja este caso internamente, pero sin retornar un indicador de error. Fue necesario consultar el estado de la lista antes de llamar a `eliminar()` para registrar correctamente si la operación fue exitosa o si ocurrió un intento sobre una lista vacía.

Corrección.

Para resolver el problema del registro del contenido de la lista, se agregó un método auxiliar `snapshot()` a la clase `ListaDinamica` en el archivo de cabecera. Este método recorre la lista y construye un string con la representación de los nodos sin imprimir nada en pantalla, siendo utilizado exclusivamente por el `Logger`. La detección de lista vacía se realizó comparando el resultado de `snapshot()` antes de llamar a `eliminar()`, registrando el evento como error si la lista estaba vacía o como exitoso en caso contrario. La lógica central de la clase no fue alterada.

Práctica 8: Pila.

Dificultades y errores.

La principal dificultad fue comprender el funcionamiento LIFO (Last In, First Out) de una pila dinámica utilizando las librerías o clases proporcionadas por el entorno de desarrollo. Fue necesario identificar correctamente las operaciones de inserción y eliminación sobre el elemento superior.

Durante las pruebas iniciales se produjeron errores al intentar acceder a elementos inexistentes cuando la pila se encontraba vacía.

Corrección.

La solución consistió en validar el contenido de la pila antes de realizar operaciones de extracción o consulta. También se implementaron pruebas de inserción y eliminación para verificar el comportamiento correcto de la estructura.

Con estas correcciones la pila funcionó adecuadamente, permitiendo manipular los elementos siguiendo el principio LIFO.

Práctica 9: Cola.

Dificultades y errores.

La principal dificultad fue adaptarse al uso de las clases proporcionadas por la biblioteca estándar para implementar una cola dinámica. Aunque la estructura ya se encontraba desarrollada internamente, fue necesario comprender el uso correcto de sus métodos.

Durante las pruebas aparecieron errores relacionados con el acceso a elementos cuando la cola estaba vacía y con la interpretación del orden de salida de los datos.

Corrección.

La solución consistió en utilizar correctamente los métodos de inserción, consulta y eliminación proporcionados por la biblioteca. También se agregaron validaciones para evitar operaciones inválidas sobre colas vacías.

Una vez realizadas estas correcciones, la estructura operó de forma eficiente y con menor complejidad de implementación.

Práctica 10: Lista Dinámica.

Dificultades y errores.

La principal dificultad fue comprender el funcionamiento de las listas dinámicas proporcionadas por las bibliotecas estándar y aprovechar correctamente sus métodos para insertar, recorrer y eliminar elementos.

Durante las primeras pruebas se produjeron errores al recorrer la lista debido al uso incorrecto de iteradores y métodos de acceso.

Corrección.

La solución consistió en utilizar adecuadamente los iteradores y las funciones propias de la biblioteca para manipular los elementos almacenados. También se realizaron pruebas de inserción, recorrido y eliminación para verificar el correcto comportamiento de la estructura.

Gracias a estas correcciones se logró administrar la información de forma dinámica y eficiente, aprovechando las ventajas de las clases ya implementadas por el entorno de desarrollo.

TERCER PARCIAL

Práctica 1: Escribir y Leer en XML datos básicos, arreglos, creados arreglos y los ADT

Dificultades y errores

La dificultad principal fue trabajar con XML sin utilizar librerías externas. Como tuve que manejar manualmente las etiquetas de apertura y cierre, fue necesario usar funciones de cadenas como `find()` y `substr()`, lo que ocasionó varios errores cuando las posiciones no se calculaban correctamente o cuando alguna etiqueta no coincidía con la esperada.

También resultó complicado guardar la información de las estructuras dinámicas como pilas, colas y listas. Debido a que están construidas mediante nodos enlazados, no era posible acceder directamente a sus elementos mediante índices, por lo que tuve que buscar una forma de obtener los datos sin modificar ni perder la información almacenada en las estructuras.

Otro aspecto que requirió mayor atención fue el manejo de objetos como `PersonaPOO` y `AutoPOO`. Para escribir y leer sus datos correctamente fue necesario acceder a cada atributo mediante sus respectivos métodos de acceso, y cualquier error en el orden de lectura podía provocar que los datos se recuperaran incorrectamente.

Corrección

Para facilitar la escritura del XML, se crearon funciones auxiliares que se encargan de abrir, cerrar y escribir etiquetas, manteniendo un formato ordenado y fácil de leer. En la parte de lectura se implementaron funciones específicas para localizar y extraer la información contenida entre las etiquetas.

Para las estructuras dinámicas se agregó el método `toArreglo()`, que permite recorrer internamente los nodos y copiar sus elementos a un arreglo temporal. De esta forma fue posible procesar la información desde la librería XML sin alterar el contenido original de las estructuras.

En el programa principal se organizaron las opciones mediante menús independientes para lectura y escritura. Esto permitió separar mejor cada proceso y mantener los datos almacenados en los diferentes archivos XML, logrando una ejecución más clara y ordenada.

Practica 2: Escribir y leer txt datos basicos, arreglos, creados arreglos y los ADT

Problema

Desarrollar un programa que permitiera capturar información mediante un arreglo y almacenarla en un archivo de texto (TXT), además de recuperar posteriormente la información guardada para mostrarla al usuario. El programa debía implementar una abstracción de datos mediante una clase y utilizar operaciones de lectura y escritura de archivos.

Desarrollo y solución

Se diseñó una clase denominada ArchivoTXT, la cual contiene un arreglo para almacenar nombres y los métodos necesarios para capturar, guardar y leer la información. Para la manipulación de archivos se utilizó la biblioteca <fstream>, implementando objetos ofstream para la escritura e ifstream para la lectura.

Se desarrolló un menú interactivo que permite al usuario capturar datos, almacenarlos en un archivo llamado personas.txt y posteriormente visualizar el contenido guardado. De esta forma, la información permanece almacenada incluso después de finalizar la ejecución del programa.

La solución implementada cumple con los principios básicos de abstracción de datos, ya que todas las operaciones relacionadas con el manejo de archivos se encuentran encapsuladas dentro de la clase.

Dificultades y errores

Durante el desarrollo se presentaron dificultades relacionadas con la apertura y cierre correcto de los archivos. En algunas pruebas iniciales la información no se almacenaba correctamente debido a errores en el manejo de los flujos de entrada y salida.

También surgieron inconvenientes al leer los datos guardados, ya que era necesario recorrer completamente el archivo línea por línea para mostrar toda la información almacenada.

Corrección

Para solucionar estos problemas se verificó la correcta apertura de los archivos antes de realizar operaciones de lectura o escritura y se implementó el cierre adecuado mediante la función close(). Además, se utilizaron ciclos de lectura con getline() para garantizar que todo el contenido del archivo fuera recuperado correctamente.

Finalmente, se realizaron pruebas de captura, almacenamiento y lectura para comprobar que los datos ingresados coincidieran exactamente con la información mostrada posteriormente desde el archivo TXT.

Práctica 3: Escribir y Leer JSON (Datos Básicos, Arreglos y ADT POO)

Problema

Implementar un programa que permitiera escribir y leer información en formato JSON utilizando datos básicos, arreglos y Programación Orientada a Objetos mediante un ADT.

Desarrollo y solución

Se desarrolló un programa utilizando los archivos main.cpp, Tipos.h y Adtjson.h. El programa permite capturar datos básicos, arreglos numéricos y objetos de tipo Alumno para almacenarlos en archivos JSON. Además, incluye opciones para leer y mostrar la información guardada.

Dificultades y errores

La principal dificultad fue construir correctamente la estructura JSON utilizando llaves, corchetes, comillas y comas. También se presentaron complicaciones al almacenar arreglos y objetos dentro de los archivos JSON, manteniendo una sintaxis válida.

Corrección

Se implementaron funciones específicas para escribir y leer datos básicos, arreglos y objetos. Finalmente, se realizaron pruebas para verificar que los archivos JSON se generaran correctamente y que la información mostrada coincidiera con la almacenada.

Práctica 4: Archivos.

Dificultades y errores.

La principal dificultad de esta práctica fue integrar en un mismo programa diferentes conceptos de programación estructurada, incluyendo la creación de tipos de datos mediante estructuras, el almacenamiento de información en archivos XML y la implementación del algoritmo de Dijkstra para la recuperación de rutas más cortas.

Uno de los inconvenientes encontrados fue la gestión de dos estructuras diferentes, **Auto** y **Persona**, ya que cada una requería un formato distinto para almacenar sus datos dentro del archivo XML. Además, al guardar información nueva, el contenido anterior era reemplazado completamente, por lo que era necesario verificar cuidadosamente qué datos estaban siendo almacenados.

Otra dificultad se presentó en la implementación del algoritmo de Dijkstra, debido a que era necesario mantener correctamente los vectores de distancias y padres para reconstruir las rutas más cortas desde el nodo origen hacia todos los demás nodos del grafo.

La principal dificultad de esta práctica fue integrar conceptos de Programación Orientada a Objetos con el manejo de archivos XML y algoritmos de grafos dentro de una misma aplicación. Fue necesario diseñar correctamente las clases **Auto** y **Persona**, encapsulando tanto los datos como las operaciones de captura, visualización y almacenamiento.

Otro problema encontrado fue asegurar que cada objeto pudiera generar correctamente su representación en formato XML mediante los métodos **guardarXML()**, respetando la estructura adecuada de las etiquetas y evitando errores durante la escritura del archivo.

Asimismo, la implementación del algoritmo de Dijkstra requirió un manejo adecuado de estructuras dinámicas como vectores y colas de prioridad. Durante las pruebas iniciales surgieron dificultades para reconstruir correctamente las rutas completas a partir de la información almacenada en el vector de padres.

Corrección.

La solución consistió en crear funciones independientes para cada tarea del programa. Se implementaron las funciones **guardarAutoXML()** y **guardarPersonaXML()** para generar correctamente la estructura XML correspondiente a cada tipo de dato. Asimismo, se desarrolló la función **mostrarXML()** para recuperar y visualizar el contenido almacenado.

Para el cálculo de rutas más cortas se utilizó una cola de prioridad junto con vectores de distancias y padres, permitiendo actualizar eficientemente cada recorrido y reconstruir posteriormente la ruta completa hacia cada nodo.

La separación de responsabilidades en funciones independientes permitió mantener una estructura organizada, facilitando el mantenimiento y la comprensión del programa.

La solución consistió en encapsular todas las operaciones relacionadas con cada entidad dentro de sus respectivas clases. Los métodos **capturar()**, **mostrar()** y **guardarXML()** permitieron que cada objeto administrara su propia información sin depender de funciones externas, fortaleciendo así los principios de encapsulamiento y modularidad.

Para la recuperación de rutas se implementó correctamente el algoritmo de Dijkstra utilizando una cola de prioridad para seleccionar los nodos con menor distancia acumulada. Además, se empleó un vector de padres para reconstruir las rutas más cortas desde el nodo origen hacia cada destino.

Gracias a esta organización orientada a objetos, el programa logró integrar el manejo de datos, persistencia en archivos XML y procesamiento de grafos de forma clara, reutilizable y fácil de mantener.

Práctica 5: Implementación de Grafos, Algoritmo de Dijkstra y Persistencia en XML (Grafo)

Dificultades y errores

El principal reto fue representar un grafo dentro de un archivo XML. A diferencia de otras estructuras más simples, un grafo requiere almacenar tanto los nodos como las conexiones existentes entre ellos, por lo que fue necesario diseñar una estructura adecuada para guardar toda esta información.

Durante la lectura del archivo surgieron problemas cuando alguna arista hacía referencia a nodos que todavía no habían sido cargados. Esto ocasionaba índices inválidos y errores al intentar actualizar la matriz de adyacencia.

En cuanto al algoritmo de Dijkstra, una de las dificultades fue manejar correctamente los nodos que no tenían conexión con el origen. Además, como las rutas se reconstruyen desde el destino hacia el origen utilizando el arreglo `prev[]`, fue necesario encontrar una forma de mostrar el recorrido en el orden correcto.

Corrección

Para solucionar estos problemas, el archivo XML se organizó en dos secciones claramente separadas: una para los nodos y otra para las aristas. Durante la lectura, primero se cargan todos los nodos y después se procesan las conexiones, evitando referencias inválidas.

La implementación de Dijkstra se realizó utilizando arreglos para almacenar las distancias, los nodos visitados y los predecesores. Para mostrar las rutas en el orden correcto se utilizó una pila temporal que permite invertir el recorrido antes de imprimirlo en pantalla.

Finalmente, se agregó una función que genera automáticamente un archivo XML de prueba con varias ciudades y conexiones. Esto facilitó las pruebas del programa y permitió verificar que tanto el almacenamiento como los algoritmos funcionaran correctamente en diferentes escenarios.

Práctica 6: Digrafo POO

Problema

Implementar un digrafo utilizando Programación Orientada a Objetos, permitiendo crear vértices, agregar aristas dirigidas y visualizar la información mediante una matriz de adyacencia. Se desarrolló una clase Digrafo para almacenar la matriz de adyacencia y la cantidad de vértices.

Además, se creó la clase DigrafoADT, encargada de administrar las operaciones principales como crear el digrafo, agregar aristas, mostrar la matriz de adyacencia y visualizar las conexiones dirigidas entre vértices.

El programa se organizó en los archivos Digrafo.h, DigrafoADT.h y main.cpp, utilizando un menú interactivo para facilitar la interacción con el usuario.

Dificultades y errores

La principal dificultad fue comprender el funcionamiento de un digrafo y la forma de representar las conexiones dirigidas mediante una matriz de adyacencia. También surgieron errores al validar los vértices ingresados por el usuario y al intentar agregar aristas antes de crear el digrafo.

Corrección

Se implementaron validaciones para verificar que el digrafo existiera antes de realizar operaciones sobre él y que los vértices ingresados estuvieran dentro del rango permitido. Finalmente, se realizaron pruebas verificando que las aristas se almacenaran correctamente y que la matriz de adyacencia representara adecuadamente las conexiones dirigidas del digrafo.

Práctica 7: Árbol con Lectura XML/JSON y Algoritmo de Dijkstra

Problema

Desarrollar un programa capaz de representar una estructura de árbol mediante conexiones entre nodos, leyendo la información desde archivos XML o JSON. El sistema debía validar cada conexión nodo a nodo, almacenar la información en memoria y aplicar el algoritmo de Dijkstra para calcular y recuperar las rutas más cortas desde un nodo de origen.

Desarrollo y solución

Se implementó una clase denominada Arbol, encargada de administrar las conexiones entre nodos mediante estructuras de datos dinámicas. El programa permite leer información desde archivos XML y JSON, validando cada conexión antes de incorporarla a la estructura interna del árbol.

Se desarrolló un menú interactivo con opciones para cargar archivos XML, cargar archivos JSON, visualizar las conexiones registradas y ejecutar el algoritmo de Dijkstra. Una vez seleccionada la opción correspondiente, el sistema calcula las distancias mínimas desde un nodo origen hacia los demás nodos y muestra las rutas completas recorridas.

La implementación utiliza listas de adyacencia para representar las conexiones y una cola de prioridad para optimizar la ejecución del algoritmo de Dijkstra, permitiendo obtener resultados eficientes incluso cuando aumenta la cantidad de nodos.

Dificultades y errores

Uno de los principales desafíos fue interpretar correctamente la información almacenada en los archivos XML y JSON. Durante las primeras pruebas se presentaron errores al extraer los valores de origen, destino y peso de cada conexión debido a diferencias en la estructura de los archivos.

También surgieron dificultades al reconstruir las rutas más cortas obtenidas por Dijkstra, ya que fue necesario almacenar información adicional sobre los nodos predecesores para poder mostrar el recorrido completo.

Otra dificultad consistió en validar adecuadamente las conexiones leídas, evitando registros incompletos o inconsistentes que pudieran afectar el cálculo de rutas.

Corrección

Para resolver estos problemas se implementaron procesos de validación durante la lectura de los archivos, verificando que cada conexión contuviera correctamente los datos de origen, destino y peso antes de ser almacenada.

Asimismo, se incorporó un arreglo de nodos predecesores que permite reconstruir y mostrar las rutas más cortas obtenidas por el algoritmo de Dijkstra. Se realizaron múltiples pruebas utilizando diferentes configuraciones de archivos XML y JSON para comprobar la exactitud de las distancias calculadas y la correcta recuperación de las rutas.

Finalmente, se verificó el funcionamiento de todas las opciones del menú interactivo, garantizando una lectura adecuada de archivos, una representación correcta de las conexiones y una ejecución confiable del algoritmo de rutas más cortas.